Q.1 Write and explain the iterative back tracking algorithm. Draw the state space tree for 4-queens problem and give the solution tuples.

ans. Iterative Backtracking Algorithm:

Iterative backtracking is a technique used to find all solutions to a problem by building the solution step by step and removing candidates that do not satisfy constraints, without using recursion. The algorithm starts by initializing an empty solution array and placing the first candidate in the first position. It then checks if this candidate satisfies the problem constraints. If it does, the algorithm moves to the next position and repeats the process. If no valid candidate is found for a position, it backtracks to the previous position and tries the next candidate. This process continues until all positions are filled or all possibilities are exhausted. When a complete valid solution is found, it is recorded. Iterative backtracking can be implemented using loops and arrays instead of recursive calls, making it easier to manage in memory.

4-Queens Problem and

Solution:

Q.1

ans. In the 4-queens problem, the goal is to place four queens on a 4×4 chessboard so that no two queens attack each other. Using iterative backtracking, we try placing a queen in each column of the first row and then move row by row, placing queens only in positions that are not attacked by previously placed queens. If a conflict occurs, we backtrack and try the next column. By following this method, we explore all possible positions in a structured manner, often represented as a state space tree where each level corresponds to a row of the chessboard and each branch represents a possible column for a queen.

The two solution tuples for the 4-queens problem are (2, 4, 1, 3) and (3, 1, 4, 2). In each tuple, the numbers indicate the column positions of queens in each row. For example, in (2, 4, 1, 3), the queen in row 1 is placed in column 2, the queen in row 2 is in column 4, the queen in row 3 is in column 1, and the queen in row 4 is in column 3. These placements ensure that no two queens share the same row, column, or diagonal.

Q.2. Define the relationship among P, NP, NP Hard and

Q.1 NP complete problems? Show with the help of a diagram. Relationship among P, NP, NP-Hard, and NP-Complete Problems:

ans. In computational complexity theory, problems are classified based on how efficiently they can be solved and verified.

P (Polynomial time): This class includes problems that can be solved by a deterministic Turing machine in polynomial time. In other words, there exists an algorithm that can find a solution efficiently.

NP (Nondeterministic Polynomial time): These are problems for which a given solution can be verified in polynomial time, even if finding the solution may be difficult. Every problem in P is also in NP because if a problem can be solved quickly, its solution can also be verified quickly.

NP-Complete: These problems are both in NP and as hard as any problem in NP. If any NP-complete problem can be solved in polynomial time, then all NP problems can also be solved in polynomial time. NP-complete problems are considered the hardest problems in

NP.

Q.1

NP-Hard: This class includes problems that are at least as hard as NP-complete problems. NP-hard problems do not have to be in NP; they may not even be decision problems. Solving an NP-hard problem efficiently would allow us to solve all NP-complete problems efficiently as well.

ans.

Q.3.Here is the question and its step-by-step solution.QuestionObtain the reduced cost matrix for the travelling salesperson problem. Consider the instance defined by the cost matrix: \begin{bmatrix}
\infty + 5 + 1 + 10 + 6 \\
1 + \infty + 4 + 12 + 7 \\
3 + 6 + \infty + 4 + 16 \\
7 + 1 + 3 + \infty + 9 \\
16 + 12 + 7 + 6 + \infty
\end{bmatrix}

Here is the question and its step-by-step solution.QuestionObtain the reduced cost matrix for the travelling salesperson problem. Consider the instance defined by the cost matrix: \begin{bmatrix}
\infty + 5 + 1 + 10 + 6 \\

Q.1 1 + \infty + 4 + 12 + 7 \\

3 + 6 + \infty + 4 + 16 \\

7 + 1 + 3 + \infty + 9 \\

ans. 16 + 12 + 7 + 6 + \infty

\end{bmatrix} Solve it by Least Cost Branch and Bound.Answer: Solving TSP using Least Cost Branch and Bound The goal is to find the shortest tour that visits every city exactly once and returns to the starting city (City 1). We will use the Least Cost Branch and Bound (LCBB) method.Step 1: Initial matrix Reduction (Root Node)First, we reduce the initial cost matrix by subtracting the minimum value from each row and then from each column. This will give us the initial lower bound (LB).Initial Cost matrix (C): C =

\begin{bmatrix}

\infty + 5 + 1 + 10 + 6 \\

1 + \infty + 4 + 12 + 7 \\

3 + 6 + \infty + 4 + 16 \\

7 + 1 + 3 + \infty + 9 \\

16 + 12 + 7 + 6 + \infty

\end{bmatrix} 1. Row Reduction:Row 1: min = 1.

Subtract 1: [\infty, 4, 0, 9, 5] Row 2: min = 1.

Subtract 1: [0, \infty, 3, 11, 6] Row 3: min = 3.

Subtract 3: [0, 3, \infty, 1, 13] Row 4: min = 1.

Q.1 ans.

Subtract 1: [6, 0, 2, \infty, 8] Row 5: min = 6.
Subtract 6: [10, 6, 1, 0, \infty] Row Reduction
Cost = 1 + 1 + 3 + 1 + 6 = 12 Row-Reduced
matrix: \begin{bmatrix}
\infty + 4 + 0 + 9 + 5 \\
0 + \infty + 3 + 11 + 6 \\
0 + 3 + \infty + 1 + 13 \\
6 + 0 + 2 + \infty + 8 \\
10 + 6 + 1 + 0 + \infty
\end{bmatrix} 2. Column Reduction:Col 1: min = 0. Cost = 0.Col 2: min = 0. Cost = 0.Col 3: min = 0. Cost = 0.Col 4: min = 0. Cost = 0.Col 5: min = 5.
Subtract 5: [0, 1, 8, 3, \infty] Column Reduction
Cost = 0 + 0 + 0 + 0 + 5 = 5 Initial Reduced
matrix (C_1): C_1 = \begin{bmatrix}
\infty + 4 + 0 + 9 + 0 \\
0 + \infty + 3 + 11 + 1 \\
0 + 3 + \infty + 1 + 8 \\
6 + 0 + 2 + \infty + 3 \\
10 + 6 + 1 + 0 + \infty
\end{bmatrix} Initial Lower Bound (Root Node 1): LB(1) = \text{Row Cost} + \text{Col Cost} = 12 + 5 = 17 Step 2: Branching from Node 1 (Level 1)We calculate the lower bound for traveling from City 1 to each of the other cities (2, 3, 4, 5). The

Q.1 formula for the lower bound of a child node k is: LB(k) = \text{Parent_LB} + C_{\text{parent_reduced}} (1, k) + \text{ReductionCost} (\text{new matrix}) Path

ans. 1 \to 2 (Node 2): LB(2) = LB(1) + C_1(1, 2) + \text{Reduction Cost} LB(2) = 17 + 4 + 3 = 24 Path 1 \to 3 (Node 3): LB(3) = LB(1) + C_1(1, 3) + \text{Reduction Cost} LB(3) = 17 + 0 + 2 = 19 Path 1 \to 4 (Node 4): LB(4) = LB(1) + C_1(1, 4) + \text{Reduction Cost} LB(4) = 17 + 9 + 2 = 28 Path 1 \to 5 (Node 5): LB(5) = LB(1) + C_1(1, 5) + \text{Reduction Cost} LB(5) = 17 + 0 + 1 = 18 State Space Tree (Level 1): LB(1 \to 2) = 24 LB(1 \to 3) = 19 LB(1 \to 4) = 28 LB(1 \to 5) = 18 Decision: The "Least Cost" path is 1 \to 5 with LB = 18 . We explore this node next.Step 3: Branching from Node 5 (Path 1 \to 5)We now branch from the path 1 \to 5 to cities 2, 3, and 4. The parent LB is 18.Path 1 \to 5 \to 2 (Node 5,2): LB(5, 2) = LB(5) + C_{\text{reduced_5}} (5, 2) + \text{Reduction Cost} LB(5, 2) = 18 + 6 + 3 = 27 Path 1 \to 5 \to 3 (Node 5,3): LB(5, 3) =

Q.1 ans.

LB(5) + C_{\text{reduced_5}} } (5, 3) + \text{Reduction Cost} LB(5, 3) = 18 + 0 + 1 = 19 Path 1 \to 5 \to 4 (Node 5,4): LB(5, 4) = LB(5) + C_{\text{reduced_5}} } (5, 4) + \text{Reduction Cost} LB(5, 4) = 18 + 0 + 1 = 19 State Space Tree (Active Nodes): LB(1 \to 3) = 19 LB(1 \to 5 \to 3) = 19 LB(1 \to 5 \to 4) = 19 LB(1 \to 2) = 24 LB(1 \to 5 \to 2) = 27 LB(1 \to 4) = 28 Decision: We have a three-way tie for the minimum LB of 19. Let's explore 1 \to 5 \to 3 .Step 4: Branching from Node (5,3) (Path 1 \to 5 \to 3)We branch from 1 \to 5 \to 3 to cities 2 and 4. The parent LB is 19.Path 1 \to 5 \to 3 \to 2 (Node 5,3,2): LB(5, 3, 2) = LB(5, 3) + C_{\text{reduced_5,3}} } (3, 2) + \text{Reduction Cost} LB(5, 3, 2) = 19 + 2 + 17 = 38 Path 1 \to 5 \to 3 \to 4 (Node 5,3,4): LB(5, 3, 4) = LB(5, 3) + C_{\text{reduced_5,3}} } (3, 4) + \text{Reduction Cost} LB(5, 3, 4) = 19 + 0 + 0 = 19 State Space Tree (Active Nodes): LB(1 \to 3) = 19 LB(1 \to 5 \to 4) = 19 LB(1 \to 5 \to 3 \to 4) = 19 ... (all other nodes have LB >

Q.1 ans.

19)Decision: We have another tie. Let's explore 1 \to 5 \to 3 \to 4 .Step 5: Branching from Node (5,3,4) (Path 1 \to 5 \to 3 \to 4)From this path, only City 2 is left to visit.Path 1 \to 5 \to 3 \to 4 \to 2:This is a complete tour. The last edge is 2 \to 1 .Let's calculate the cost of this tour from the original cost matrix: C(1, 5) = 6 C(5, 3) = 7 C(3, 4) = 4 C(4, 2) = 1 C(2, 1) = 1 Total Cost = 6 + 7 + 4 + 1 + 1 = 19 We have found a complete tour with a cost of 19. This cost becomes our first Upper Bound (UB). UB = 19 Step 6: Pruning the TreeNow we check all other active (unexplored) nodes. We can "prune" (discard) any node where its LB is greater than or equal to our new UB . LB(1 \to 3) = 19 .(LB \geq UB). Prune. LB(1 \to 5 \to 4) = 19 .(LB \geq UB). Prune. LB(1 \to 2) = 24 .(LB > UB). Prune. LB(1 \to 5 \to 2) = 27 . (LB > UB). Prune. LB(1 \to 4) = 28 . (LB > UB). Prune. LB(1 \to 5 \to 3 \to 2) = 38 .(LB > UB). Prune.Since all other promising paths have been explored or pruned, the first solution we found is the optimal one.Final Solution The algorithm terminates. The optimal solution is the path